Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 25

Перенос веб-приложения на Next.js для реализации SSR и SSG

Вариант 2 – лендинг IT-конференции

Выполнила: Клычева Г.
Группа: 310102
Репозиторий: https://github.com/karrinkl/devconf-next.git
Проверил: Прудник А.М.

1 ЦЕЛЬ РАБОТЫ И УСЛОВИЕ ЗАДАНИЯ

Цель работы состоит в получении практических навыков создания производительных и SEO-дружественных веб-приложений с помощью мета-фреймворка. Требуется перенести React-приложение, созданное в предыдущих лабораторных работах, на Next.js и реализовать гибридную стратегию рендеринга: часть страниц формируется сервером при каждом запросе, часть генерируется заранее во время сборки.

Для варианта 2, лендинга IT-конференции, условие задаёт две ключевые страницы. Страница «Расписание» по адресу /schedule должна рендериться на сервере, чтобы всегда показывать актуальный список докладов и их статусы. Страница «Спикеры» по адресу /speakers генерируется статически, поскольку список спикеров утверждается заранее и меняется редко.

Исходной точкой служит проект лабораторной работы № 23, лендинг DEVCONF 2026 на React, React Bootstrap и Redux Toolkit. Из него перенесены компоненты, данные восьми спикеров, тема оформления поверх Bootstrap и срез состояния «Избранное». Сверх условия добавлены страницы отдельных спикеров /speakers/[id], чтобы показать генерацию динамических маршрутов функцией getStaticPaths.

Технические требования задания, которые проверяются в работе:

– проект создан на Next.js, ранее написанные компоненты лежат в каталоге components;
– маршруты формируются файловым роутингом из каталога pages;
– страница с SSR получает данные функцией getServerSideProps, которая возвращает текущую дату и время;
– страница с SSG получает данные функцией getStaticProps во время сборки, для динамического маршрута реализована getStaticPaths;
– переходы между страницами выполняются компонентом Link;
– приложение собирается командой npm run build и запускается командой npm run start, а исходный код страниц в браузере содержит готовый HTML.

Выбран Next.js, поскольку проект написан на React. Использована версия 16 с маршрутизатором Pages Router: именно в нём определены функции getServerSideProps, getStaticProps и getStaticPaths, которые требует задание.

2 СТРАТЕГИИ РЕНДЕРИНГА

Одностраничное приложение из предыдущих работ использовало рендеринг на стороне клиента (CSR). Сервер Vite отдавал почти пустой файл index.html с элементом div id="root", а всю разметку строил JavaScript уже в браузере. Поисковый робот, который не исполняет скрипты, видел пустую страницу, а пользователь видел содержимое только после загрузки и выполнения всего бандла.

Next.js добавляет две стратегии, при которых HTML формируется заранее на сервере. Их отличия от клиентского рендеринга сведены в таблицу 1.

Таблица 1 – Сравнение стратегий рендеринга

Признак	CSR	SSR	SSG
Когда строится HTML	В браузере после загрузки JavaScript	На сервере при каждом запросе	Один раз во время сборки
Функция в Next.js	Нет, данные загружаются в useEffect	getServerSideProps	getStaticProps, для динамических путей ещё getStaticPaths
Содержимое в исходном коде страницы	Пустой контейнер	Полная разметка на момент запроса	Полная разметка на момент сборки
Актуальность данных	Всегда свежие	Всегда свежие	Устаревают до следующей сборки
Нагрузка на сервер	Минимальная	Вычисления на каждый запрос	Отдача готового файла
Индексация поисковиками	Затруднена	Полная	Полная
Применение в проекте	Интерактив: избранное, форма	Расписание со статусами	Спикеры и страницы спикеров

Выбор стратегии определяется тем, как часто меняются данные. Статусы докладов зависят от текущего времени, поэтому расписание формируется на каждый запрос. Состав спикеров фиксирован, и пересчитывать его для каждого посетителя незачем: страница собирается один раз и отдаётся как файл.

После получения готового HTML страница проходит гидратацию: React подключает обработчики событий к уже существующей разметке. Поэтому звёздочки «в избранное» и форма регистрации продолжают работать так же, как в ЛР № 23, а хранилище Redux живёт на клиенте.

3 ПЕРЕНОС ПРОЕКТА И ФАЙЛОВЫЙ РОУТИНГ

Проект создан заново по структуре create-next-app, после чего в него перенесены исходные файлы ЛР № 23. Соответствие файлов двух проектов приведено в таблице 2.

Таблица 2 – Перенос файлов из проекта Vite в Next.js

Vite, ЛР № 23	Next.js, ЛР № 25
index.html	pages/_document.js: язык страницы, иконка вкладки
src/main.jsx	pages/_app.js: глобальные стили, Provider, общий Layout
src/App.jsx	pages/index.js и отдельные файлы страниц
src/components/*	components/*: SpeakerCard, Favorites, Icon, RegistrationForm
src/assets/*	public/icons и public/images, путь вида /images/speaker-01-1x.webp
src/store/index.js	store/index.js: фабрика makeStore вместо единственного объекта
src/data/content.js	data/speakers.js, data/faq.js, data/schedule.js
src/styles/theme.css	styles/theme.css, подключается только в _app.js

Импорт изображений как модулей заменён путями к каталогу public: Next.js раздаёт его содержимое по корневому адресу без обработки. Глобальные таблицы стилей в Next.js разрешено подключать только в _app.js, поэтому импорт Bootstrap и темы перенесён туда, а CSS-модуль компонента Icon остался рядом с компонентом. Шрифт Inter подключён из пакета @fontsource/inter, и сайт открывается с правильной типографикой без обращения к Google Fonts.

Отдельного внимания потребовало хранилище Redux. В Vite-проекте оно создавалось один раз при загрузке модуля. На сервере модуль загружается один раз на весь процесс, и такое хранилище стало бы общим для всех посетителей. Поэтому модуль экспортирует функцию makeStore, а _app.js вызывает её через useState: на сервере получается новый объект для каждого запроса, в браузере один объект на всю жизнь вкладки.

```
export default function App({ Component, pageProps }) {
  const [store] = useState(makeStore);
  return (
    <Provider store={store}>
      <Layout>
        <Component {...pageProps} />
      </Layout>
    </Provider>
  );
}
```

Маршруты задаются расположением файлов в каталоге pages, отдельная библиотека маршрутизации не нужна. Полученная карта сайта приведена в таблице 3.

Таблица 3 – Маршруты приложения

Адрес	Файл	Стратегия	Содержимое
/	pages/index.js	Static	Первый экран, FAQ, регистрация
/schedule	pages/schedule.js	SSR	Таблица докладов со статусами
/speakers	pages/speakers/index.js	SSG	Сетка карточек спикеров
/speakers/[id]	pages/speakers/[id].js	SSG	Страница спикера, 8 путей
любой другой	pages/404.js	Static	Страница «не найдено»

Главная страница не экспортирует функций получения данных, поэтому Next.js применяет к ней автоматическую статическую оптимизацию и тоже собирает её в HTML заранее. Квадратные скобки в имени [id].js объявляют параметр маршрута, значение которого приходит в функции страницы через context.params.

Навигация собрана в компоненте Layout. Пункты меню Nav.Link из React Bootstrap отрисовываются как Link из next/link через свойство as. Ссылка сохраняет оформление Bootstrap, а переход выполняется на клиенте без перезагрузки: Next.js подгружает только JSON с props новой страницы. Отмеченные звёздочкой спикеры при таких переходах сохраняются, потому что хранилище не пересоздаётся.

```
<Nav.Link as={Link} href="/schedule" active={isActive("/schedule")}>
  Расписание
</Nav.Link>
```

4 СЕРВЕРНЫЙ РЕНДЕРИНГ СТРАНИЦЫ «РАСПИСАНИЕ»

Страница /schedule экспортирует асинхронную функцию getServerSideProps. Next.js вызывает её на сервере при каждом запросе, дожидается результата, отрисовывает компонент страницы с полученными props и отправляет браузеру готовый HTML. Код функции и импортированные ею модули в клиентский бандл не попадают.

```
export async function getServerSideProps({ res }) {
  const now = Date.now();

  // имитация обращения к источнику данных
  const schedule = buildSchedule(now);

  // ответ нельзя кешировать: нужны свежие статусы
  res.setHeader('Cache-Control', 'no-store');

  return {
    props: {
      schedule,
      renderedAt: formatTimestamp(new Date(now)),
      liveCount: schedule.filter((i) => i.status === 'live').length,
    },
  };
}
```

Источник данных сымитирован модулем data/schedule.js. Время начала каждого доклада в нём задано не абсолютной датой, а смещением в минутах от момента запроса: значение -30 означает «начался полчаса назад», значение 25 означает «начнётся через 25 минут». Функция buildSchedule переводит смещения во время по часовому поясу Минска и вычисляет статус доклада.

```
export function getStatus(startMs, endMs, now) {
  if (now >= endMs) return { code: 'finished', label: 'Завершён' };
  if (now >= startMs) return { code: 'live', label: 'Идёт сейчас' };
  if (startMs - now <= SOON_WINDOW * MINUTE)
    return { code: 'soon', label: 'Скоро' };
  return { code: 'planned', label: 'Запланирован' };
}
```

Благодаря такому приёму расписание всегда выглядит «живым»: в нём есть завершённые доклады, доклады в эфире и ближайшие. Отметка времени renderedAt выводится в плашке над таблицей компонентом RenderInfo. Форматирование выполняется на сервере, а в props передаются только строки и числа, поскольку Next.js сериализует props в JSON для гидратации. Внешний вид страницы приведён на рисунке 1.

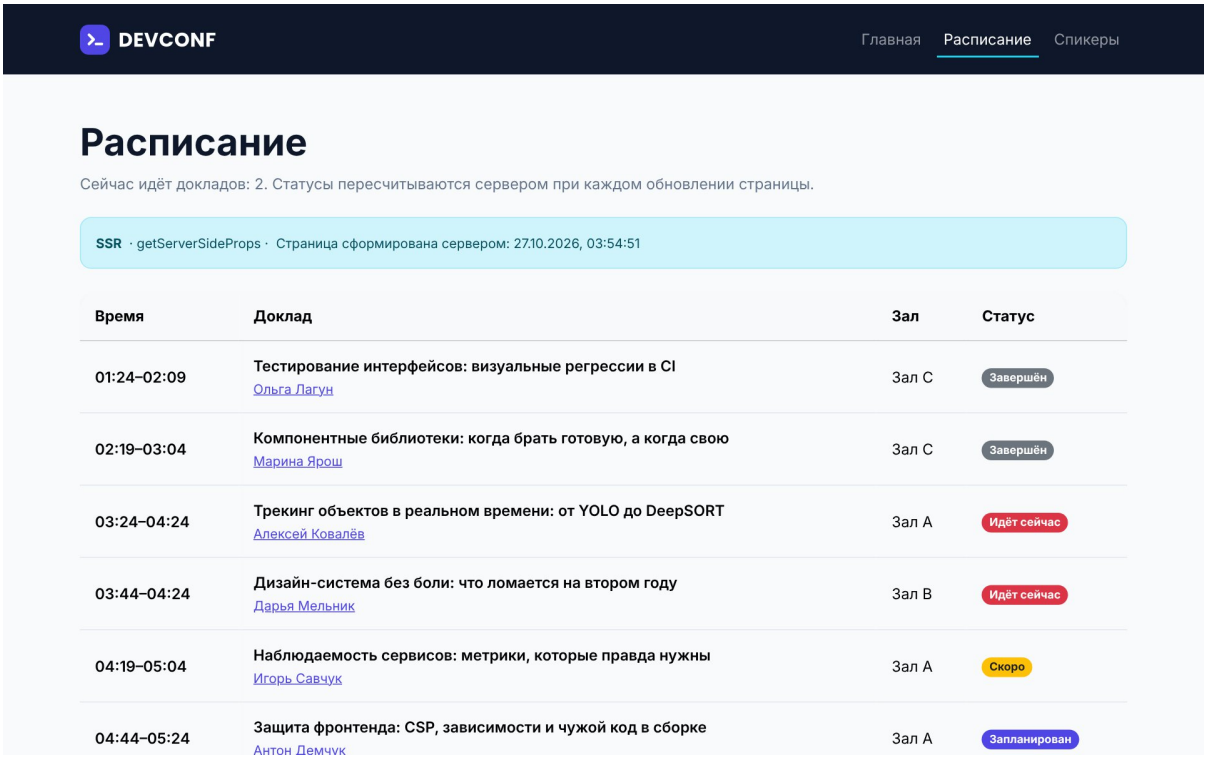


Рисунок 1 – Страница «Расписание», сформированная сервером

Красными метками отмечены доклады, которые идут в момент запроса, жёлтой меткой ближайший доклад. При обновлении страницы время в плашке и статусы пересчитываются, без пересборки проекта.

5 СТАТИЧЕСКАЯ ГЕНЕРАЦИЯ СТРАНИЦ СПИКЕРОВ

Страница /speakers экспортирует функцию getStaticProps. В production-режиме Next.js вызывает её один раз при выполнении npm run build и сохраняет результат в готовый файл speakers.html. Сервер отдаёт этот файл всем посетителям без повторных вычислений.

```
export async function getStaticProps() {
  // в браузер уходят только поля, нужные карточке
  const list = speakers.map(({ bio, tags, ...card }) => card);

  return {
    props: {
      speakers: list,
      builtAt: formatTimestamp(),
    },
  };
}
```

Функция отбрасывает поля bio и tags, которые на странице списка не выводятся. Это уменьшает объём JSON, встроенного в HTML для гидратации. Отметка builtAt фиксирует момент сборки и не меняется до следующего запуска npm run build. Страница приведена на рисунке 2.

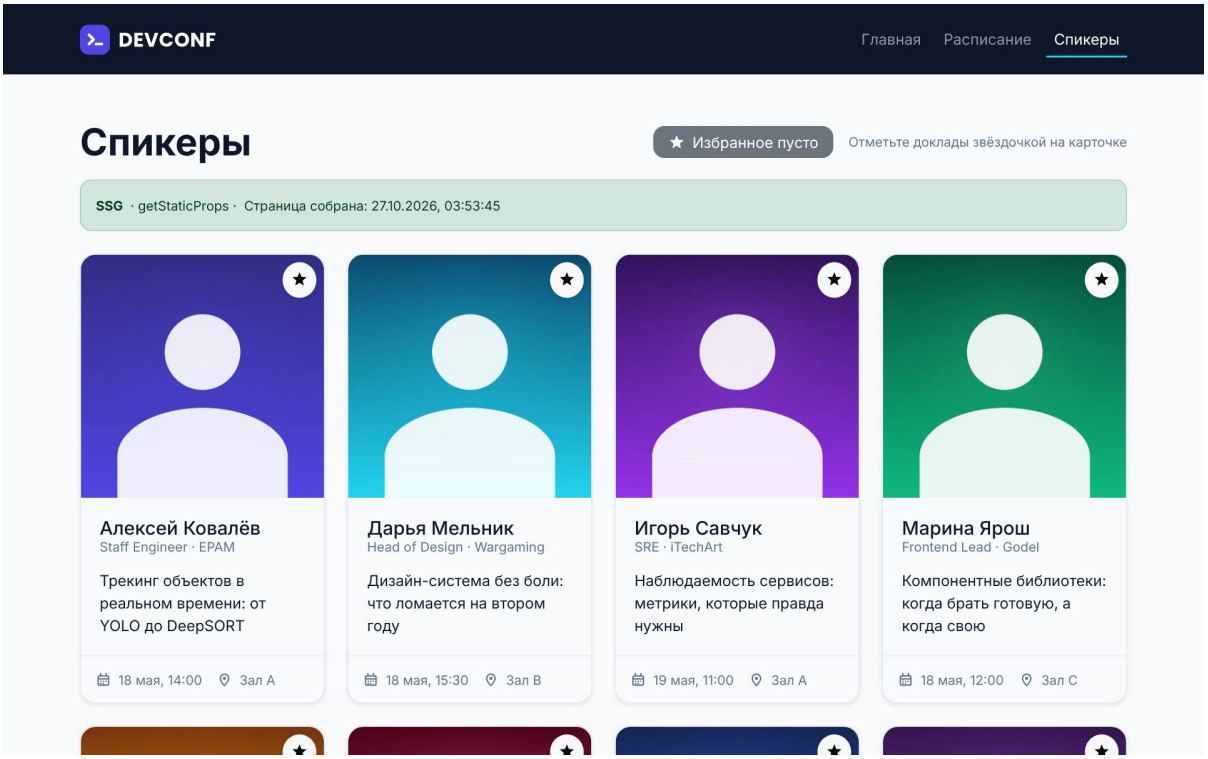


Рисунок 2 – Страница «Спикеры», сгенерированная при сборке

Для страниц отдельных спикеров маршрут динамический: в адресе /speakers/[id] значение id заранее неизвестно фреймворку. Чтобы сгенерировать такие страницы при сборке, Next.js вызывает getStaticPaths и получает список всех допустимых значений параметра.

```
export async function getStaticPaths() {
  return {
    paths: speakers.map((s) => ({ params: { id: s.id } })),
    fallback: false,
  };
}

export async function getStaticProps({ params }) {
  const speaker = getSpeakerById(params.id);
  if (!speaker) return { notFound: true };
  return { props: { speaker, builtAt: formatTimestamp() } };
}
```

Для каждого из восьми путей вызывается getStaticProps с соответствующим params.id, и при сборке появляются файлы kovalev.html, melnik.html и остальные. Значение fallback: false означает, что других страниц нет: запрос /speakers/unknown сразу получает ответ 404. Страница спикера приведена на рисунке 3.

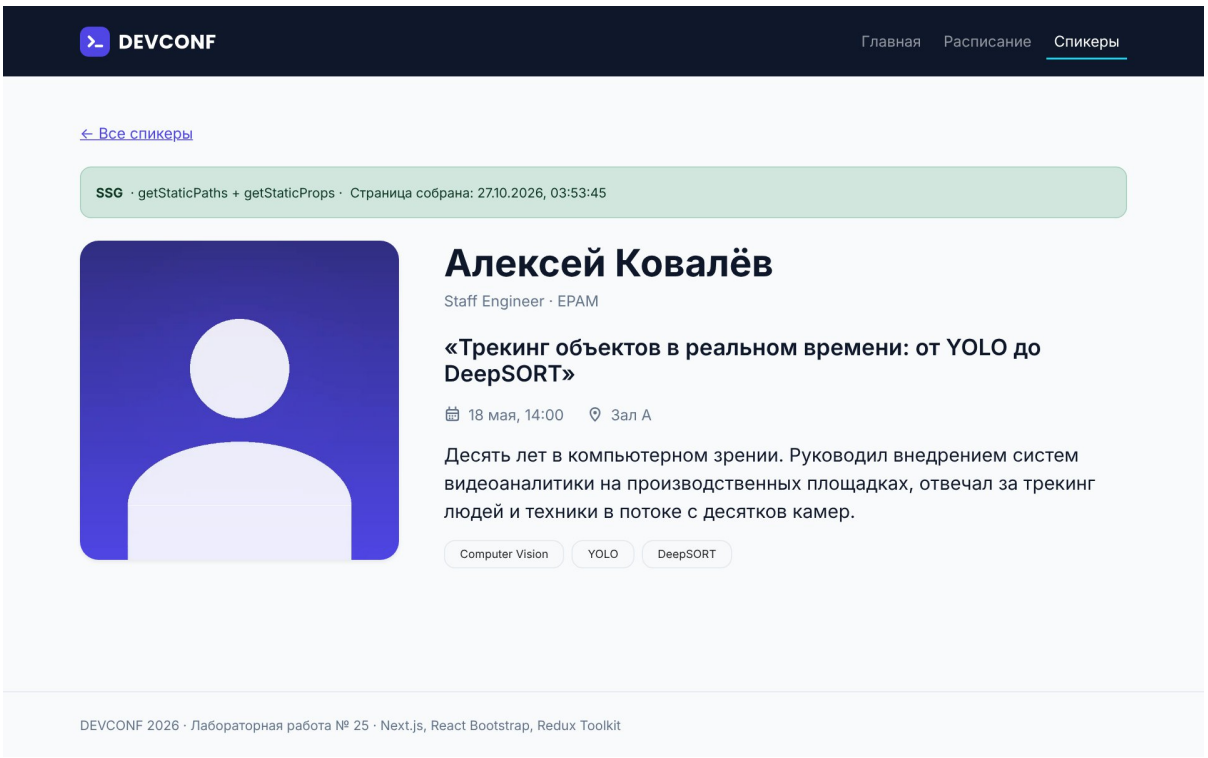


Рисунок 3 – Страница спикера /speakers/kovalev

На странице спикера выводятся биография и теги, которые на странице списка были отброшены. Отметка времени совпадает с отметкой на /speakers: обе страницы собраны одной командой npm run build.

6 ПРОВЕРКА РАБОТЫ SSR И SSG

Проверка выполнялась в production-режиме: в режиме разработки Next.js вызывает getStaticProps на каждый запрос, и разница между стратегиями не видна. Результат команды npm run build приведён на рисунке 4.

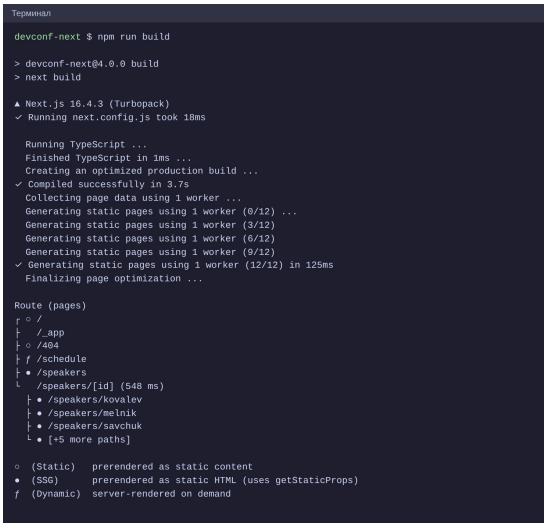


Рисунок 4 – Вывод команды npm run build

В таблице маршрутов Next.js сам указывает стратегию каждой страницы. Значок ○ у / и /404 означает статическую страницу без данных, значок ● у /speakers и восьми путей /speakers/[id] означает SSG с getStaticProps, значок *ƒ* у /schedule означает серверный рендеринг по запросу. Всего сгенерировано 12 статических страниц. Готовые HTML-файлы, созданные при сборке, показаны на рисунке 5.

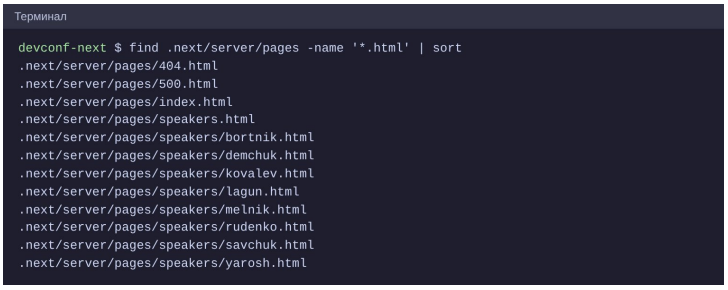


Рисунок 5 – Файлы статических страниц в каталоге .next

Файла schedule.html среди них нет: эта страница не существует до запроса и формируется сервером каждый раз заново. Исходный код страницы /speakers, открытый сочетанием клавиш Ctrl+U, приведён на рисунке 6. Искомые фрагменты выделены.

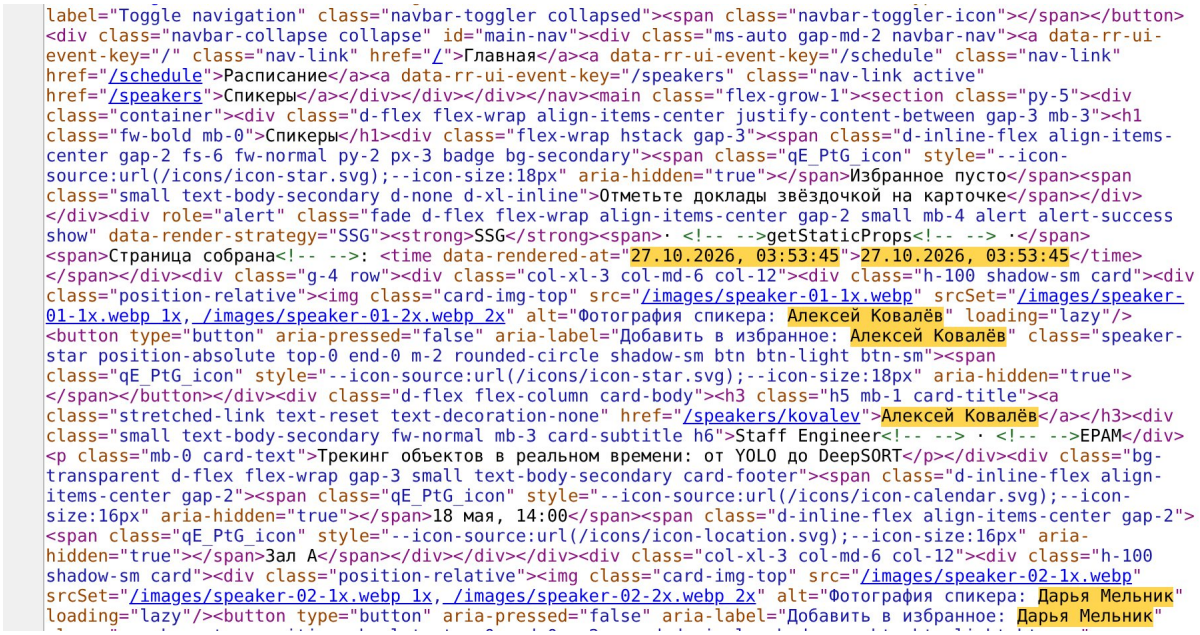


Рисунок 6 – Исходный код страницы /speakers

В HTML, пришедшем от сервера, уже есть имена спикеров, темы докладов, пути к фотографиям и плашка «Страница собрана: 27.10.2026, 03:53:45». Браузер получает готовую разметку, а не пустой контейнер, как в CSR-версии проекта. При повторных запросах отметка времени остаётся прежней, потому что сервер отдаёт один и тот же файл.

Исходный код страницы /schedule приведён на рисунках 7 и 8. Второй снимок сделан после повторного запроса страницы через несколько секунд.



Рисунок 7 – Исходный код страницы /schedule, первый запрос



Рисунок 8 – Исходный код страницы /schedule, повторный запрос

В обоих случаях таблица докладов и статусы «Завершён», «Идёт сейчас» уже присутствуют в HTML. Отметка времени отличается: 04:02:10 при первом запросе и 04:02:14 при втором. Это подтверждает, что getServerSideProps выполняется на сервере при каждом обращении к странице, а не один раз при сборке.

Те же выводы следуют из заголовков HTTP-ответов, полученных командой curl и приведённых на рисунке 9.

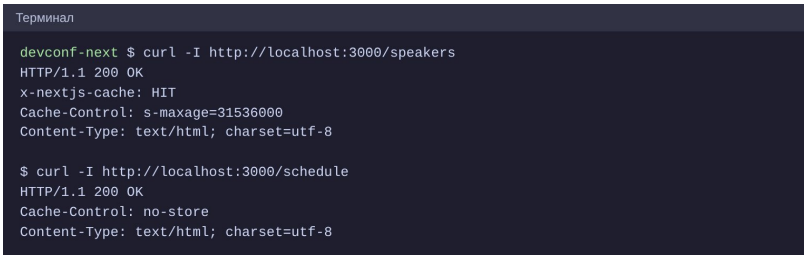


Рисунок 9 – Заголовки ответов для /speakers и /schedule

Для /speakers сервер возвращает заголовок x-nextjs-cache: HIT, то есть страница взята из готового кеша сборки, и разрешает кешировать её в CDN на год. Для /schedule сервер возвращает Cache-Control: no-store, заданный в getServerSideProps, и промежуточные кеши не сохраняют ответ.

Отдельно проверена клиентская навигация: на странице /speakers спикер отмечен звёздочкой, затем выполнены переходы на /schedule и обратно через меню. Счётчик избранного сохранил значение «В избранном: 1 спикер», ошибок гидратации в консоли браузера нет.

Результаты всех проверок сведены в таблицу 4.

Таблица 4 – Результаты проверки стратегий рендеринга

Проверка	Ожидаемый результат	Фактический результат
Таблица маршрутов npm run build	/schedule помечена *ƒ*, /speakers и /speakers/[id] помечены ●	Совпадает, 12 статических страниц
Файлы в .next/server/pages	Есть speakers.html и 8 файлов спикеров, нет schedule.html	Совпадает
Ctrl+U на /speakers	Имена спикеров и время сборки в HTML	Есть, 27.10.2026, 03:53:45
Повторный Ctrl+U на /speakers	Время не меняется	Не меняется
Ctrl+U на /schedule	Таблица со статусами и время запроса в HTML	Есть, 04:02:10
Повторный Ctrl+U на /schedule	Время меняется	04:02:14
Заголовки ответа	HIT для /speakers, no-store для /schedule	Совпадает
Запрос /speakers/unknown	Ответ 404	404
Переходы через Link	Без перезагрузки, избранное сохраняется	Совпадает, ошибок в консоли нет

Все проверки дали ожидаемый результат, расхождений между заявленной и фактической стратегией рендеринга не обнаружено.

7 СБОРКА И ЗАПУСК

Для запуска нужен Node.js версии 20.9 или новее. В Arch Linux Node.js устанавливается из официального репозитория командой sudo pacman -S nodejs npm. Последовательность команд для production-режима:

```
git clone https://github.com/karrinkl/devconf-next.git
cd devconf-next
npm install
npm run build
npm run start
```

После запуска сайт открывается по адресу http://localhost:3000. Для разработки используется команда npm run dev с горячей перезагрузкой. Эти инструкции, таблица маршрутов и порядок проверки SSR и SSG продублированы в файле README.md репозитория.

ЗАКЛЮЧЕНИЕ

Лендинг DEVCONF 2026 перенесён с Vite на Next.js 16. Компоненты, данные, тема Bootstrap и хранилище Redux из ЛР № 23 разложены по структуре мета-фреймворка, маршруты формируются файловым роутингом из каталога pages, навигация выполнена компонентом Link.

Реализована гибридная стратегия рендеринга. Страница «Расписание» формируется сервером на каждый запрос функцией getServerSideProps и показывает статусы докладов на момент обращения. Страница «Спикеры» и восемь страниц отдельных спикеров генерируются при сборке функциями getStaticProps и getStaticPaths и отдаются как готовые файлы.

Проверка исходного кода страниц в браузере, вывод сборки и заголовки ответов подтвердили, что HTML для обеих стратегий приходит с сервера, отметка времени SSR-страницы меняется при каждом запросе, а у SSG-страниц остаётся равной моменту сборки. Приложение собирается командой npm run build и запускается командой npm run start.